zkDietFormer: Verifiable Transformer Inference with Linear-Cost Attention Proofs via Low-Rank Approximation

First Author[1], Second Author[2], and Third Author[3]

[1] Institute 1
[2] Institute 2
[3] Institute 3

Abstract. The widespread deployment of Transformer-based models via Model-as-a-Service platforms raises a fundamental trust problem: users cannot verify that the provider faithfully executed the claimed model on their input. Zero-knowledge SNARKs (zk-SNARKs) can bridge this gap by letting the provider prove correct inference without revealing proprietary model weights, yet existing approaches face an $O(n^2)$ proving-cost bottleneck rooted in the $n \times n$ matrix products and row-wise softmax of the quadratic self-attention mechanism.

We propose **zkDietFormer**, the first zero-knowledge verification framework designed for low-rank self-attention. By instantiating the Nyström approximation inside the proving circuit, we replace the full $n \times n$ attention computation, including the nonlinear cost induced by the row-wise softmax, with three smaller matrix products involving only $m \ll n$ landmark tokens, reducing the dominant cost to $O(n)$. The prover supplies the Nyström pseudoinverse as a witness that the circuit constrains via the Moore–Penrose conditions; Freivalds' randomized checks verify all large matrix products, and a commit-carrying SNARK offloads the weight and intermediate-matrix commitments outside the circuit, binding every response to a single committed model.

Experiments on five BERT-scale configurations (hidden dimensions 128–768, depths 2–12) show that zkDietFormer extends the maximum provable sequence length by up to 4× over standard attention under the same 128 GB memory budget, with per-layer prove time growing near-linearly in n.

Keywords: Zero-Knowledge Proofs · Verifiable AI · Low-Rank Attention · Transformer · zk-SNARKs

1 Introduction

Recent breakthroughs in Transformer-based models have driven the widespread deployment of high-performance Large Language Models (LLMs) via Model-as-a-Service (MaaS) platforms, where users receive outputs through remote inference APIs. This architecture inherently obscures the generation process: users

cannot verify whether the provider substituted a cheaper model, altered outputs before returning them, or silently changed the model during an update. This problem of model execution integrity is particularly critical in high-stakes domains such as finance, healthcare, and security.

Disclosing model parameters for verification is not a viable remedy: model weights embody substantial investments in data, compute, and engineering, and are protected as proprietary intellectual property. A fundamental dilemma thus arises: users demand integrity guarantees for the model's outputs, while providers must preserve the privacy of their parameters.

Zero-knowledge proofs (ZKPs) provide a principled way to reconcile these conflicting requirements. By leveraging ZKPs, a prover (the model provider) can cryptographically prove that an output y is the result of evaluating the model on the user's input x with a secret weight W, i.e., $f(x, W) = y$, without revealing any information about W itself. This enables verifiable inference while maintaining the confidentiality of model parameters.

However, making ZKP-based verifiable AI practically deployable requires efficiently proving computationally intensive deep learning operations, and in modern Transformer architectures the primary bottleneck is self-attention. For an input sequence of length n, self-attention forms an $n \times n$ attention matrix and applies a row-wise softmax over its entirety, incurring $\mathcal{O}(n^2 d)$ linear operations plus an additional $\mathcal{O}(n^2)$ nonlinear cost (Section 3.1).

Probabilistic techniques such as Freivalds' algorithm [11] and Thaler's randomized checks [25] reduce matrix-multiplication verification from $\mathcal{O}(n^2 d)$ to $\mathcal{O}(n^2)$, yet the verification cost remains anchored at $\mathcal{O}(n^2)$. This quadratic term becomes increasingly prohibitive as context windows grow: BERT [9] uses 512 tokens and GPT-3 [2] 2,048, while frontier models now reach 400K (GPT-5 [20]) and 10M (Llama 4 Scout [19]). As n grows, quadratic verification cost renders proof generation for realistic sequence lengths impractical.

Meanwhile, attention matrices often exhibit approximate low-rank structure: with $m \ll n$ landmarks the attention matrix admits a rank-m approximation that reduces inference cost to $\mathcal{O}(n)$ [27, 30]. Crucially, this efficiency does not come at the cost of predictive quality: Nyströmformer reports accuracy comparable to standard self-attention on the GLUE and Long Range Arena benchmarks [30], making low-rank Transformers a practical deployment choice and their inference verification a problem of immediate relevance. Yet, to the best of our knowledge, verifiable-AI research has not exploited this structure to reduce proof generation costs, keeping proof complexity at $\mathcal{O}(n^2)$. We bridge this gap by combining low-rank approximation with probabilistic verification to eliminate the quadratic bottleneck in verifying self-attention.

1.1 Our Contribution

In this work, we propose **zkDietFormer**, a verification protocol that eliminates the dominant $\mathcal{O}(n^2)$ bottleneck in proving self-attention by combining a low-rank representation of the attention matrix, Freivalds' randomized checks, and a commit-carrying SNARK. Our main contributions are summarized as follows:

- **The first verification framework tailored to low-rank self-attention.** To the best of our knowledge, no prior verifiable-inference work exploits the low-rank structure of attention. We instantiate our framework with the Nyström method, reducing the circuit complexity of attention verification—including both the core matrix products and the nonlinear operations induced by row-wise softmax—to $\mathcal{O}(n)$ in the sequence length.
- **Constraint-efficient verification via Freivalds' checks and CC-SNARKs.** All large matrix products are computed outside the circuit and verified inside it through Freivalds' randomized projections, so the circuit performs only vector inner products. A commit-carrying SNARK generates the commitments to the externally supplied witnesses—the weights and the intermediate matrices—outside the circuit, binding every proof to the committed weights and intermediates while significantly reducing the constraint count.
- **Practical implementation and evaluation.** We implement and benchmark **zkDietFormer** across five BERT-family configurations. Per-layer prove time grows near-linearly in n, and our combined optimizations extend the maximum provable sequence length by up to $4\times$ under a fixed 128 GB memory budget.

2 Related Works

Low-Rank Transformer. Standard Transformer attention [26] has $\mathcal{O}(n^2)$ complexity. Sparse patterns [6, 1] and memory compression [21] reduce cost but sacrifice full attention. Low-rank methods restore linear complexity while approximating the full attention map: Linformer [27] projects keys and values into a lower-dimensional space; Performer [7] approximates softmax via random-feature kernelization; and Nyströmformer [30] reconstructs the attention map from m landmark tokens via the Nyström method, reporting higher accuracy than Performer on standard benchmarks with $\mathcal{O}(n)$ complexity.

zk-SNARKs for Matrix Multiplication. Thaler [25] reduces matrix-multiplication verification to $\mathcal{O}(n^2)$ via Sumcheck; zkCNN [18] and zkML [5] adopt this for neural-network layers, the latter using Freivalds' algorithm [11]. vCNN [17] represents convolutions as Quadratic Polynomial Programs; pvCNN [28] extends this to Quadratic Matrix Programs, casting a convolution layer as a single matrix product. zkLLM [24] achieves prover complexity linear in tensor size via specialized sumcheck protocols, building on SafetyNets [12]. zkMatrix [8] uses Bulletproofs-based acceleration for $\mathcal{O}(n^2)$ prover time with $\mathcal{O}(\log n)$ verification. However, all these approaches retain $\mathcal{O}(n^2)$ prover cost in the matrix dimension [25, 18, 17, 28, 5, 8] or, in the case of zkLLM [24], remain linear in the size of the $n \times n$ attention tensor and hence quadratic in the sequence length, and nonlinear functions such as softmax require approximation [31]. None of these lines of work exploits the low-rank structure of attention on the proving side; **zkDietFormer** closes this gap by verifying a Nyström-approximated attention map directly.

3 Preliminaries

Notation. We denote the sequence length by n, the hidden dimension by d, and the per-head query/key and value dimensions by d_k and d_v, respectively. The number of attention heads is h, the number of Transformer layers is L, and the number of landmarks is m, giving a segment length $\ell = n/m$ (we assume m divides n). The circuit precision parameter k_{bits} controls the bit-width of quantized fixed-point arithmetic.

3.1 Self-Attention

Given an input $\mathbf{X} \in \mathbb{R}^{n\times d}$, queries, keys, and values are obtained via learned projections $\mathbf{Q} = \mathbf{X}\mathbf{W}^Q$, $\mathbf{K} = \mathbf{X}\mathbf{W}^K$, $\mathbf{V} = \mathbf{X}\mathbf{W}^V$ with $\mathbf{W}^Q, \mathbf{W}^K \in \mathbb{R}^{d\times d_k}$, $\mathbf{W}^V \in \mathbb{R}^{d\times d_v}$. Scaled dot-product attention is

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right)\mathbf{V}.$$

In multi-head attention, h heads independently compute attention with per-head dimensions $d_k = d_v = d/h$, and their outputs are concatenated and projected by $\mathbf{W}^O \in \mathbb{R}^{d\times d}$. For clarity, we describe the protocol for a single head with $d_k = d_v = d$; the extension to h heads follows by applying the same construction per head.

3.2 Nyström-Based Linear Attention

Nyströmformer applies the Nyström method [30]—a technique widely used in matrix approximation—to reduce the computational complexity of the standard Transformer from $\mathcal{O}(n^2)$ to $\mathcal{O}(n)$. The key idea of the Nyström method is to reconstruct the full matrix approximately by using only a subset of its rows and columns, instead of computing the entire matrix explicitly.

When the softmax matrix S is expressed as a block matrix as shown below,

$$S = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right) = \begin{bmatrix} A_S & B_S \\ F_S & C_S \end{bmatrix}$$

the Nyström method approximates the expensive block $C_S \in \mathbb{R}^{(n-m)\times(n-m)}$ by using the smaller blocks $A_S \in \mathbb{R}^{m\times m}, B_S \in \mathbb{R}^{m\times(n-m)}, F_S \in \mathbb{R}^{(n-m)\times m}$ as illustrated below:

$$\hat{S} = \begin{bmatrix} A_S & B_S \\ F_S & F_S A_S^+ B_S \end{bmatrix} = \begin{bmatrix} A_S \\ F_S \end{bmatrix} A_S^+ \begin{bmatrix} A_S & B_S \end{bmatrix}$$

Here, A_S^+ denotes the Moore-Penrose pseudoinverse of A_S.

However, when this approach is applied directly to self-attention, the row-wise normalization property of the softmax function still requires accessing all entries of the $n \times n$ matrix in order to compute the denominators. To address

this issue, Nyströmformer partitions the entire sequence into m segments of length $\ell = n/m$ before applying softmax, and computes the mean within each segment to obtain m landmark points for the queries and keys, respectively. These landmarks yield $\widetilde{\mathbf{Q}}, \widetilde{\mathbf{K}} \in \mathbb{R}^{m \times d}$, from which three low-rank factors $\mathbf{Q}\widetilde{\mathbf{K}}^\top \in \mathbb{R}^{n \times m}$, $\widetilde{\mathbf{Q}}\widetilde{\mathbf{K}}^\top \in \mathbb{R}^{m \times m}$, and $\widetilde{\mathbf{Q}}\mathbf{K}^\top \in \mathbb{R}^{m \times n}$ are formed. The resulting factors play the same structural role as $\begin{pmatrix} A_S \\ F_S \end{pmatrix}$, A_S^+, and $\begin{pmatrix} A_S & B_S \end{pmatrix}$ in the block decomposition above, but are computed from landmarks rather than extracted from S. Using these matrices, Nyströmformer derives a final approximation that achieves linear computational complexity:

$$\hat{S}V = \mathrm{softmax}\left(\frac{\mathbf{Q}\widetilde{\mathbf{K}}^\top}{\sqrt{d_k}}\right) \mathrm{softmax}\left(\frac{\widetilde{\mathbf{Q}}\widetilde{\mathbf{K}}^\top}{\sqrt{d_k}}\right)^+ \mathrm{softmax}\left(\frac{\widetilde{\mathbf{Q}}\mathbf{K}^\top}{\sqrt{d_k}}\right) V$$

The exact computation of the pseudoinverse A_S^+ (e.g., via SVD) is computationally expensive and poorly suited for GPU execution, becoming a practical bottleneck in implementation. To mitigate this issue, Nyströmformer introduces an iterative algorithm that approximates the pseudoinverse using repeated matrix multiplications. The resulting approximation Z^* is then substituted for the pseudoinverse in the formulation above, significantly improving computational efficiency.

3.3 zk-SNARKs

A *zero-knowledge succinct non-interactive argument of knowledge* (zk-SNARK) for an NP relation $\mathcal{R}$ consists of three PPT algorithms $\Pi = (\mathsf{Setup}, \mathsf{Prove}, \mathsf{Verify})$: $(\mathsf{pk}, \mathsf{vk}) \leftarrow \Pi.\mathsf{Setup}(1^\lambda, \mathcal{R})$ outputs a proving key and verification key; $\pi \leftarrow \Pi.\mathsf{Prove}(\mathsf{pk}, \mathbf{x}; \mathbf{w})$ produces a proof for statement $\mathbf{x}$ and witness $\mathbf{w}$ with $(\mathbf{x}, \mathbf{w}) \in \mathcal{R}$; and $0/1 \leftarrow \Pi.\mathsf{Verify}(\mathsf{vk}, \mathbf{x}, \pi)$ accepts or rejects. A zk-SNARK satisfies *completeness* (an honest prover always convinces the verifier), *knowledge soundness* (an accepting proof implies the prover knows a valid witness), *zero-knowledge* (the proof reveals nothing beyond the statement's truth), and *succinctness* (the proof size is $|\pi| = poly(\lambda) \cdot polylog(|\mathbf{x}| + |\mathbf{w}|)$). Formal definitions appear in [13].

Commit-and-Prove SNARKs (CP-SNARKs). CP-SNARKs [4, 3] let a prover demonstrate that values hidden inside a commitment $\mathsf{cm} = \mathsf{Com}(\mathbf{u}; o)$ satisfy a relation $\mathcal{R}$. The witness is split as $\mathbf{w} := (\omega, \mathbf{u}, o)$, where $\mathbf{u}$ is committed and ω is not. The interface mirrors a plain SNARK except that Setup additionally takes a commitment key ck, Prove takes cm as part of the statement and includes $(\mathbf{u}, o)$ in the witness, and Verify checks both the relation and the commitment consistency.

Commit-Carrying SNARKs (CC-SNARKs). CC-SNARKs [4, 15] extend CP-SNARKs by generating the commitment *during* proving rather than beforehand. The setup outputs a circuit-dependent commitment key ck alongside $(\mathsf{pk}, \mathsf{vk})$. The prover partitions the witness as $\mathbf{w} := (\mathbf{u}, \omega)$ and outputs both a proof π_{cc} and a commitment $\tilde{\mathsf{cm}} = \mathsf{Com.Commit}(\mathsf{ck}, \mathbf{u}; \tilde{o})$. The verifier

checks $\Pi_{\mathsf{cc}}.\mathsf{Verify}(\mathsf{vk}, \mathbf{x}, \tilde{\mathsf{cm}}, \pi_{\mathsf{cc}})$. This design moves expensive group exponentiations outside the circuit, which is critical for scaling to large Transformer weight matrices.

3.4 Freivalds' Algorithm

Freivalds' algorithm [11, 16] is a simple yet powerful randomized method for verifying a matrix product without recomputing it. Let $A, B, C \in \mathbb{F}^{n\times n}$ and consider the claim $AB = C$. Instead of computing AB in $\mathcal{O}(n^3)$ time, Freivalds samples a uniformly random vector $\mathbf{r} \xleftarrow{\$} \mathbb{F}^n$ and checks

$$A(B\mathbf{r}) \stackrel{?}{=} C\mathbf{r}.$$

This verification runs in $\mathcal{O}(n^2)$ field operations and has one-sided error: if $AB = C$, the check always accepts; otherwise it passes with probability at most $1/|\mathbb{F}|$, i.e.,

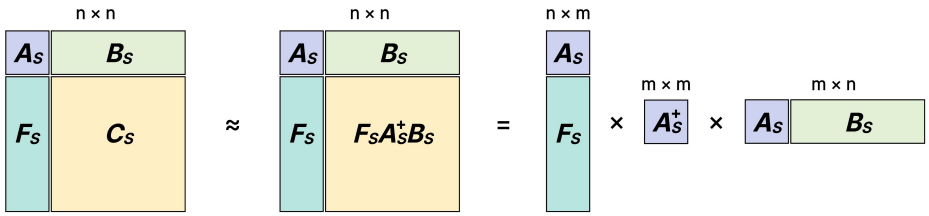
$$\Pr\big[\, A(B\mathbf{r}) = C\mathbf{r} \;\big|\; AB \neq C \,\big] \leq \frac{1}{|\mathbb{F}|}.$$

The same principle extends to rectangular matrices and to the transposed (left-multiplied) variant $\mathbf{r}^\top AB \stackrel{?}{=} \mathbf{r}^\top C$, which we use in Section 5 to verify products of varying dimensions.

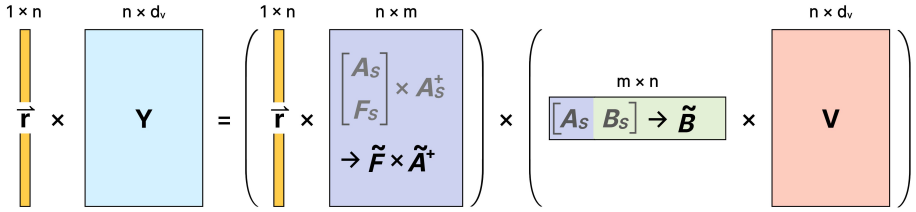
4 Technical Overview

Among the low-rank attention mechanisms reviewed in Section 2, we adopt Nyströmformer [30] because it requires no additional learned projection matrices in the proving circuit and provides numerically stable reconstruction regardless of sequence length. We exclude Performer [7], whose FAVOR+ random feature map introduces nonlinear exponential evaluations that would increase circuit complexity.

As illustrated in Figure 1a, rather than directly handling the $n \times n$ self-attention matrix, we follow the Nyströmformer procedure and decompose it into a sequence of low-dimensional matrices of shapes $n \times m$, $m \times m$, and $m \times n$. We denote the three low-rank factors by $\tilde{F} := \mathrm{softmax}(\widetilde{\mathbf{Q}}\widetilde{\mathbf{K}}^\top/\sqrt{d_k}) \in \mathbb{F}^{n\times m}$, $\tilde{A} := \mathrm{softmax}(\widetilde{\mathbf{Q}}\widetilde{\mathbf{K}}^\top/\sqrt{d_k}) \in \mathbb{F}^{m\times m}$, and $\tilde{B} := \mathrm{softmax}(\tilde{\mathbf{Q}}\mathbf{K}^\top/\sqrt{d_k}) \in \mathbb{F}^{m\times n}$, treated formally in Section 5.2. In the block decomposition of Figure 1a, $\tilde{F}$, $\tilde{A}^+$, and $\tilde{B}$ take the roles of $\left[\begin{smallmatrix} A_S \\ F_S \end{smallmatrix}\right]$, A_S^+, and $\left[\begin{smallmatrix} A_S & B_S \end{smallmatrix}\right]$, respectively; they are computed from the landmarks before the softmax rather than extracted from S (cf. Section 3.2), so Figure 1b is obtained from Figure 1a by this substitution. By applying Freivalds' algorithm to this low-rank representation (Figure 1b), we compress the verification workload from $\mathcal{O}(n^2)$ to $\mathcal{O}(n)$, effectively reducing it to a chain of vector–matrix products of total size $2nm + m^2 + nd_v$: multiplying the random vector by $\tilde{F}$ (nm), then by $\tilde{A}^+$ (m^2), then by $\tilde{B}$ (mn), and finally by $\mathbf{V}$ (nd_v). In the formal relation, this chain is verified in three fixed-point



(a) Nyströmformer matrix approximation



(b) Our **zkDietFormer** Freivalds' Check

Fig. 1: Low-rank matrix multiplication with Freivalds' algorithm

rescaled stages, with the two intermediate products committed (Section 5.3). The pseudoinverse factor is instantiated by the iterative approximation Z^* of Section 3.2: the prover computes Z^* outside the circuit and supplies it as a witness, and our protocol verifies in-circuit that Z^* satisfies the Moore–Penrose conditions (Section 5.3) up to quantization tolerance. Since these conditions uniquely characterize the pseudoinverse $\tilde{A}^+$, this check constrains Z^* to a small neighborhood of $\tilde{A}^+$, without ever computing $\tilde{A}^+$ itself.

We embed these low-rank checks in a commit-carrying SNARK (CC-SNARK) architecture (Figure 2): model weights are committed once under a circuit-independent key, and each proof carries a circuit-dependent commitment linked to the global one via CP-linkage, so the verifier needs only constant-time commitment checks (details in Section 5). Because zero-knowledge circuits operate over finite fields, real-valued Transformer computations cannot be represented directly. We therefore quantize the computations in a manner that minimizes precision loss, leveraging the R1CS-friendly quantization technique proposed in [10]. From this point onward, all matrices are understood to be over the finite field $\mathbb{F}$ after quantization.

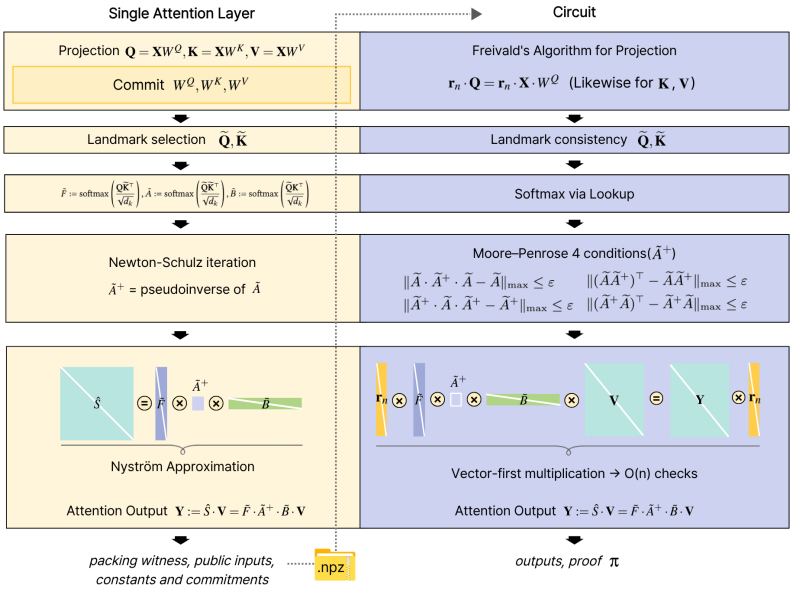


Fig. 2: Overview of Self-Attention and our framework

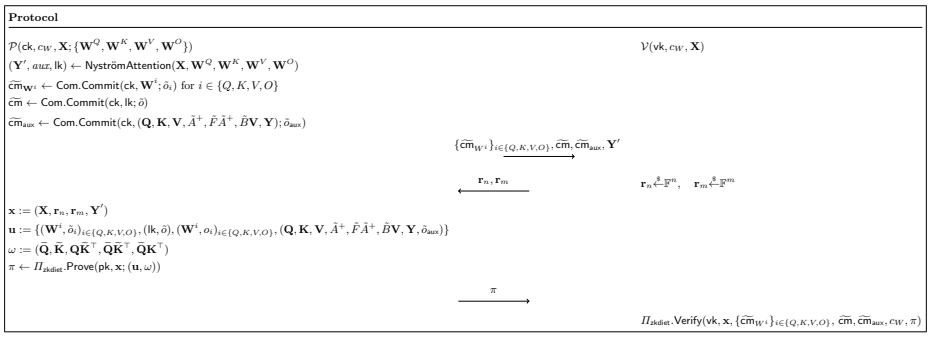


Fig. 3: Our protocol

5 zkDietFormer: Efficient Proofs of Low-Rank Matrix Multiplication via Freivalds' Algorithm

In this section, we present the core protocol of **zkDietFormer**. Our goal is to prove the correctness of Nyström-approximated low-rank self-attention while avoiding any explicit computation of the $n \times n$ attention matrix inside the circuit. Concretely, we design the protocol so that the number of constraints scales linearly with the sequence length n. To this end, we adopt a commit-carrying SNARK (CC-SNARK) architecture that pushes computationally expensive operations (e.g., large matrix multiplications and pseudoinverse computation) out-

side the circuit, and proves only their correctness inside the circuit using a small number of constraints.

5.1 Protocol Overview

Our protocol follows the message flow of Figure 3. Prior to serving any query, the provider publishes a global commitment $c_W \leftarrow \mathsf{Com.Commit}(\mathsf{ck}_{\text{glob}}, \mathbf{W}; o)$ to its weights $\mathbf{W} = (\mathbf{W}^Q, \mathbf{W}^K, \mathbf{W}^V, \mathbf{W}^O)$ under a circuit-independent key $\mathsf{ck}_{\text{glob}}$.

For each query with input $\mathbf{X} \in \mathbb{F}^{n\times d}$, the prover executes $\mathsf{NyströmAttention}$ locally to obtain $\mathbf{Y}'$, non-linear operands $\mathsf{lk} := (\tilde{F}, \tilde{A}, \tilde{B})$, and auxiliary intermediates *aux*. It then commits to the weights and operands under the circuit-dependent key ck, producing $\widetilde{\mathsf{cm}}_{\mathbf{W}^i}$ and $\widetilde{\mathsf{cm}}$[4], and to $(\mathbf{Q}, \mathbf{K}, \mathbf{V}, \tilde{A}^+, \tilde{F}\tilde{A}^+, \tilde{B}\mathbf{V}, \mathbf{Y})$ via $\widetilde{\mathsf{cm}}_{\mathsf{aux}}$. Here the juxtaposed symbols $\tilde{F}\tilde{A}^+$ and $\tilde{B}\mathbf{V}$ name the committed rescaled stage products (Figure 4). All commitments are sent together with $\mathbf{Y}'$ to the verifier. The verifier replies with random challenges $\mathbf{r}_n \xleftarrow{\$} \mathbb{F}^n$, $\mathbf{r}_m \xleftarrow{\$} \mathbb{F}^m$.

The public instance is $\mathbf{x} := (\mathbf{X}, \mathbf{r}_n, \mathbf{r}_m, \mathbf{Y}')$. The committed witness $\mathbf{u}$ contains the weight matrices and their opening randomness, the non-linear operands, and the auxiliary matrices $(\mathbf{Q}, \mathbf{K}, \mathbf{V}, \tilde{A}^+, \tilde{F}\tilde{A}^+, \tilde{B}\mathbf{V}, \mathbf{Y})$; the uncommitted witness $\omega := (\widetilde{\mathbf{Q}}, \widetilde{\mathbf{K}}, \mathbf{Q}\widetilde{\mathbf{K}}^\top, \widetilde{\mathbf{Q}}\widetilde{\mathbf{K}}^\top, \widetilde{\mathbf{Q}}\mathbf{K}^\top)$ requires no direct commitment: each entry of the three pre-softmax products is determined by an exact in-circuit equality constraint from the lookup index and remainder-limb decomposition used for the e^x lookup, both of which are included in the lookup operand commitment; $\widetilde{\mathbf{Q}}$ and $\widetilde{\mathbf{K}}$ are uniquely determined from the committed $\mathbf{Q}$ and $\mathbf{K}$ by exact in-circuit segment-averaging constraints. All commitments are fixed before challenges are sampled, preserving Freivalds soundness.

The prover generates a composite proof π that encapsulates the CC-SNARK proof together with two CP-linkages binding the circuit-dependent commitments $\{\widetilde{\mathsf{cm}}_{\mathbf{W}^i}\}$ to the global c_W and $\widetilde{\mathsf{cm}}$ to the lookup arguments. In the non-interactive variant, the challenges $\mathbf{r}_n, \mathbf{r}_m$ are derived as $H(c_W, \{\widetilde{\mathsf{cm}}_{\mathbf{W}^i}\}_i, \widetilde{\mathsf{cm}}, \widetilde{\mathsf{cm}}_{\mathsf{aux}}, \mathbf{x})$ via the Fiat–Shamir transformation.

Throughout, we write $\tilde{A}^+$ for the pseudoinverse witness, with the understanding that it is the iterative approximation Z^* of Section 3.2, verified via Moore–Penrose conditions (Section 5.3).

5.2 Nyström-Based Low-Rank Self-Attention

We instantiate the Nyström approximation of Section 3.2 inside the proving circuit. Using the landmark matrices $\widetilde{\mathbf{Q}}, \widetilde{\mathbf{K}} \in \mathbb{F}^{m\times d}$ (computed by segment-averaging as described in Section 3.2 and formalized in lines 9–12 of Figure 4), we define the three softmax factors

$$\tilde{F} := \mathrm{softmax}\Big(\tfrac{\mathbf{Q}\widetilde{\mathbf{K}}^\top}{\sqrt{d_k}}\Big), \quad \tilde{A} := \mathrm{softmax}\Big(\tfrac{\widetilde{\mathbf{Q}}\widetilde{\mathbf{K}}^\top}{\sqrt{d_k}}\Big), \quad \tilde{B} := \mathrm{softmax}\Big(\tfrac{\widetilde{\mathbf{Q}}\mathbf{K}^\top}{\sqrt{d_k}}\Big).$$

[4] The lookup operand commitment $\widetilde{\mathsf{cm}}$ covers not only the softmax operands $(\tilde{F}, \tilde{A}, \tilde{B})$ but also the range-check operands: lookup indices and the fixed-point quotient-remainder decomposition's remainder limbs, including the division remainders arising from the segment-averaging computation of the landmarks.

Algorithm 1 Nyström approximation of attention matrix

Input: Input sequence $\mathbf{X} \in \mathbb{F}^{n \times d}$,
Model parameters $\mathbf{W}^Q, \mathbf{W}^K, \mathbf{W}^V, \mathbf{W}^O \in \mathbb{F}^{d \times d}$

Output: Attention output $\mathbf{Y}' \in \mathbb{F}^{n \times d}$,
Auxiliary intermediates aux,
Non-linear operands $\mathsf{lk} := (\tilde{F}, \tilde{A}, \tilde{B})$

$\mathsf{NyströmAttention}(\mathbf{X}, \mathbf{W}^Q, \mathbf{W}^K, \mathbf{W}^V, \mathbf{W}^O)$:

$\mathbf{Q} := \mathbf{X}\mathbf{W}^Q, \quad \mathbf{K} := \mathbf{X}\mathbf{W}^K, \quad \mathbf{V} := \mathbf{X}\mathbf{W}^V$

$\widetilde{\mathbf{Q}}, \widetilde{\mathbf{K}} \leftarrow \text{SegmentAverage}(\mathbf{Q}, \mathbf{K}, m)$ (cf. Section 3.2)

$\tilde{F} := \text{softmax}\left(\frac{\mathbf{Q}\widetilde{\mathbf{K}}^\top}{\sqrt{d_k}}\right), \quad \tilde{A} := \text{softmax}\left(\frac{\widetilde{\mathbf{Q}}\widetilde{\mathbf{K}}^\top}{\sqrt{d_k}}\right), \quad \tilde{B} := \text{softmax}\left(\frac{\widetilde{\mathbf{Q}}\mathbf{K}^\top}{\sqrt{d_k}}\right)$

$\tilde{A}^+ \leftarrow \mathsf{ComputeMPInverse}(\tilde{A})$

$\mathbf{Y} := (\tilde{F} \cdot \tilde{A}^+)(\tilde{B} \cdot \mathbf{V})$

$\mathbf{Y}' := \mathbf{Y} \cdot \mathbf{W}^O$

$aux := (\mathbf{Q}, \mathbf{K}, \mathbf{V}, \widetilde{\mathbf{Q}}, \widetilde{\mathbf{K}}, \mathbf{Q}\widetilde{\mathbf{K}}^\top, \widetilde{\mathbf{Q}}\widetilde{\mathbf{K}}^\top, \widetilde{\mathbf{Q}}\mathbf{K}^\top, \tilde{A}^+, \tilde{F}\tilde{A}^+, \tilde{B}\mathbf{V}, \mathbf{Y})$

$\mathsf{lk} := (\tilde{F}, \tilde{A}, \tilde{B})$

return $(\mathbf{Y}', aux, \mathsf{lk})$

The pseudoinverse $\tilde{A}^+$ is approximated via the iterative procedure of Algorithm 2 (Appendix A). The final attention output and output projection are $\mathbf{Y} := (\tilde{F} \cdot \tilde{A}^+)(\tilde{B} \cdot \mathbf{V})$ and $\mathbf{Y}' := \mathbf{Y} \cdot \mathbf{W}^O$. Algorithm 1 gives the complete procedure.

5.3 Efficient Verification via Freivalds' Algorithm

Our protocol uses Freivalds' algorithm as a core tool to efficiently verify that the low-rank self-attention computation is performed correctly. In particular, within the zk-SNARK circuit, we employ Freivalds' algorithm to ensure that the following key operations can be verified with linear cost.

Query/Key/Value matrix generation and output projection. Using the verifier's random vector $\mathbf{r}_n$, we check the relation $\mathbf{Q} = \mathbf{X}\mathbf{W}^Q$ by comparing $\mathbf{r}_n^\top \mathbf{Q}$ with $\mathbf{r}_n^\top (\mathbf{X}\mathbf{W}^Q)$. We verify $\mathbf{K}$ and $\mathbf{V}$ in the same way. Rather than proving the full $n \times d$ matrix product directly, this Freivalds' check reduces verification to computing only the vector-matrix product $\mathbf{r}_n^\top \mathbf{X}\mathbf{W}^Q$, thereby reducing the complexity from $\mathcal{O}(nd^2)$ to $\mathcal{O}(nd + d^2)$. The output projection $\mathbf{Y}' = \mathbf{Y}\mathbf{W}^O$ is verified analogously: we check $\mathbf{r}_n^\top \mathbf{Y}' \stackrel{?}{=} (\mathbf{r}_n^\top \mathbf{Y})\,\mathbf{W}^O$.

Low-rank matrix operations. We also verify low-dimensional products such as $\mathbf{Q}\widetilde{\mathbf{K}}^\top$, $\widetilde{\mathbf{Q}}\widetilde{\mathbf{K}}^\top$, and $\widetilde{\mathbf{Q}}\mathbf{K}^\top$ by multiplying each matrix with a random vector of

matching dimension, i.e., $\mathbf{r}_n$ or $\mathbf{r}_m$. For instance, we can certify the correctness of $\mathbf{Q}\widetilde{\mathbf{K}}^\top$ by checking the following equality:

$$\mathbf{r}_n^\top\big(\mathbf{Q}\widetilde{\mathbf{K}}^\top\big) \stackrel{?}{=} \big(\mathbf{r}_n^\top\mathbf{Q}\big)\,\widetilde{\mathbf{K}}^\top$$

Similarly, the products $\widetilde{\mathbf{Q}}\widetilde{\mathbf{K}}^\top$ and $\widetilde{\mathbf{Q}}\mathbf{K}^\top$ are validated with linear cost, without explicitly computing the full matrix multiplications inside the circuit.

Moore–Penrose pseudoinverse. Computing $\tilde{A}^+$ inside the circuit is inefficient. Instead, the prover supplies $\tilde{A}^+$ as a witness, and the circuit verifies the four Moore–Penrose conditions (lines 19–22 of Figure 4) as element-wise range checks $\|\cdot\|_{\max} \le \varepsilon$, where the tolerance ε is determined by the Newton–Schulz iteration; the concrete value is specified in Section 7.1 (exact in-circuit computation is negligible: $\mathcal{O}(m^3)$ field operations for m=32).

The relations that the prover must satisfy are formalized in Figure 4 as the definition of the circuit relation. In particular, the relation $\mathcal{R}(\mathbf{x}, \mathbf{w})$ takes the public instance $\mathbf{x} = (\mathbf{X}, \mathbf{r}_n, \mathbf{r}_m, \mathbf{Y}')$ and the witness $\mathbf{w} = (\mathbf{u}, \omega)$, where the committed part $\mathbf{u}$ includes the weight matrices, the lookup operands, and the auxiliary matrices $(\mathbf{Q}, \mathbf{K}, \mathbf{V}, \tilde{A}^+, \tilde{F}\tilde{A}^+, \tilde{B}\mathbf{V}, \mathbf{Y})$, while the uncommitted part $\omega = (\widetilde{\mathbf{Q}}, \widetilde{\mathbf{K}}, \mathbf{Q}\widetilde{\mathbf{K}}^\top, \widetilde{\mathbf{Q}}\widetilde{\mathbf{K}}^\top, \widetilde{\mathbf{Q}}\mathbf{K}^\top)$ consists of values derived in-circuit from committed data. The relation enforces all required equalities as well as the softmax computation; every matrix-product identity is expressed as a projection with the random vectors, so these checks reduce to vector inner products, while the Moore–Penrose conditions are enforced element-wise on $m \times m$ matrices at negligible cost.

6 Security

In this section, we formalize the security guarantees provided by **zkDietFormer**. We first outline the system and threat models (Section 6.1), then present our security definition and delineate the attack vectors that fall outside our scope (Section 6.2).

6.1 System and Threat Model

Our system involves two parties, a *model provider* and a *user*. The provider holds the trained per-layer weight matrices $(\mathbf{W}^i)_{i\in\{Q,K,V,O\}}$ of a Nyström-based Transformer and exposes an inference interface: the user submits an input sequence $\mathbf{X} \in \mathbb{F}^{n\times d}$ and receives the corresponding output $\mathbf{Y}'$. The architectural parameters $\mathsf{zkdiet} = (n, d, h, L, m, k_{\text{bits}})$, including the number of landmarks m and the quantization precision, are public and fixed by the circuit; only the weight values are private. Before serving any query, the provider publishes a single commitment c_W to its weights under a circuit-independent key, so that every subsequent response, including responses produced under a circuit instantiated for a different sequence length, is bound to one and the same model.

Relation

$\mathcal{R}(\mathbf{x}, \mathbf{w})$:

parse $\mathbf{x} = (\mathbf{X}, \mathbf{r}_n, \mathbf{r}_m, \mathbf{Y}'), \mathbf{w} = (\mathbf{u}, \omega)$

parse $\mathbf{u} := \{(\mathbf{W}^i, \tilde{o}_i)_{i\in\{Q,K,V,O\}}, (\mathsf{lk}, \tilde{o}), (\mathbf{W}^i, o_i)_{i\in\{Q,K,V,O\}}, (\mathbf{Q}, \mathbf{K}, \mathbf{V}, \tilde{A}^+, \tilde{F}\tilde{A}^+, \tilde{B}\mathbf{V}, \mathbf{Y}, \tilde{o}_{\mathsf{aux}})\}$

parse $\omega := (\widetilde{\mathbf{Q}}, \widetilde{\mathbf{K}}, \mathbf{Q}\widetilde{\mathbf{K}}^\top, \widetilde{\mathbf{Q}}\widetilde{\mathbf{K}}^\top, \widetilde{\mathbf{Q}}\mathbf{K}^\top)$

parse $\mathbf{Q} = [\mathbf{q}_1, \mathbf{q}_2, \ldots, \mathbf{q}_n]^\top, \mathbf{K} = [\mathbf{k}_1, \mathbf{k}_2, \ldots, \mathbf{k}_n]^\top$

$\mathbf{r}_n^\top \cdot \mathbf{Q} = \mathbf{r}_n^\top \cdot \mathbf{X} \cdot \mathbf{W}^Q$

$\mathbf{r}_n^\top \cdot \mathbf{K} = \mathbf{r}_n^\top \cdot \mathbf{X} \cdot \mathbf{W}^K$

$\mathbf{r}_n^\top \cdot \mathbf{V} = \mathbf{r}_n^\top \cdot \mathbf{X} \cdot \mathbf{W}^V$

for $j \in \{1, \ldots, m\}$ **do**

$\tilde{\mathbf{q}}_j \leftarrow \sum_{i=(j-1)\times\ell+1}^{j\ell} \frac{\mathbf{q}_i}{\ell}, \tilde{\mathbf{k}}_j \leftarrow \sum_{i=(j-1)\times\ell+1}^{j\ell} \frac{\mathbf{k}_i}{\ell}$

endfor

$\widetilde{\mathbf{Q}} \leftarrow [\tilde{\mathbf{q}}_1, \ldots, \tilde{\mathbf{q}}_m]^\top, \widetilde{\mathbf{K}} \leftarrow [\tilde{\mathbf{k}}_1, \ldots, \tilde{\mathbf{k}}_m]^\top$

$\mathbf{r}_n^\top \cdot \mathbf{Q}\widetilde{\mathbf{K}}^\top = \mathbf{r}_n^\top \cdot \mathbf{Q} \cdot \widetilde{\mathbf{K}}^\top$

$\mathbf{r}_m^\top \cdot \widetilde{\mathbf{Q}}\widetilde{\mathbf{K}}^\top = \mathbf{r}_m^\top \cdot \widetilde{\mathbf{Q}} \cdot \widetilde{\mathbf{K}}^\top$

$\mathbf{r}_m^\top \cdot \widetilde{\mathbf{Q}}\mathbf{K}^\top = \mathbf{r}_m^\top \cdot \widetilde{\mathbf{Q}} \cdot \mathbf{K}^\top$

$\tilde{F} = \mathrm{softmax}\left(\frac{\mathbf{Q}\widetilde{\mathbf{K}}^\top}{\sqrt{d_k}}\right)$

$\tilde{A} = \mathrm{softmax}\left(\frac{\widetilde{\mathbf{Q}}\widetilde{\mathbf{K}}^\top}{\sqrt{d_k}}\right)$

$\tilde{B} = \mathrm{softmax}\left(\frac{\widetilde{\mathbf{Q}}\mathbf{K}^\top}{\sqrt{d_k}}\right)$

$\|\tilde{A} \cdot \tilde{A}^+ \cdot \tilde{A} - \tilde{A}\|_{\max} \le \varepsilon$

$\|\tilde{A}^+ \cdot \tilde{A} \cdot \tilde{A}^+ - \tilde{A}^+\|_{\max} \le \varepsilon$

$\|(\tilde{A}\tilde{A}^+)^\top - \tilde{A}\tilde{A}^+\|_{\max} \le \varepsilon$

$\|(\tilde{A}^+\tilde{A})^\top - \tilde{A}^+\tilde{A}\|_{\max} \le \varepsilon$

$\mathbf{r}_m^\top \cdot \tilde{B}\mathbf{V} = \mathbf{r}_m^\top \cdot \tilde{B} \cdot \mathbf{V}$

$\mathbf{r}_n^\top \cdot \tilde{F}\tilde{A}^+ = \mathbf{r}_n^\top \cdot \tilde{F} \cdot \tilde{A}^+$

$\mathbf{r}_n^\top \cdot \mathbf{Y} = \mathbf{r}_n^\top \cdot (\tilde{F}\tilde{A}^+) \cdot (\tilde{B}\mathbf{V})$

$\mathbf{r}_n^\top \cdot \mathbf{Y}' = \mathbf{r}_n^\top \cdot \mathbf{Y} \cdot \mathbf{W}^O$

Fig. 4: Relation of **zkDietFormer**. All projected checks are performed over fixed-point quotient-remainder decompositions: each left-hand symbol denotes the rescaled quotient of the corresponding product, and the truncation remainders are range-checked to $[0, 2^{k_{\mathrm{bits}}})$.

We denote the resulting functionality by $\mathbf{Y}' \leftarrow \mathcal{F}^{\varepsilon}_{\mathsf{zkdiet}}(\mathbf{W}, \mathbf{X})$, where $\mathcal{F}^{\varepsilon}_{\mathsf{zkdiet}}$ is the quantized Nyström attention pipeline of Figure 4, and ε is the residual tolerance admitted for the pseudoinverse factor.

A zero-knowledge verifiable low-rank attention inference (zkDIET) scheme is a tuple of algorithms $\mathsf{zkDIET} = (\mathsf{KeyGen}, \mathsf{Setup}, \mathsf{Commit}, \mathsf{Prove}, \mathsf{Verify})$. We separate key generation into two algorithms because the two commitment layers of our protocol serve different purposes: one binds the model across its entire lifetime, the other binds a single proof.

- $\mathsf{ck}_{\mathrm{glob}} \leftarrow \mathsf{zkDIET.KeyGen}(1^\lambda, N)$: given the security parameter λ and an upper bound N on the number of field elements to be committed, output a circuit-independent commitment key. This key is sampled once and is shared by every circuit configuration the provider may later deploy.
- $(\mathsf{ck}, \mathsf{pk}, \mathsf{vk}) \leftarrow \mathsf{zkDIET.Setup}(1^\lambda, \mathcal{R}_{\mathsf{zkdiet}})$: given the relation induced by the architectural parameters zkdiet, output a circuit-dependent commitment key ck together with a proving key and a verification key. This algorithm is run once per architecture, not per query.
- $c_W \leftarrow \mathsf{zkDIET.Commit}(\mathsf{ck}_{\mathrm{glob}}, \mathbf{W}; o)$: commit to the weight matrices $\mathbf{W} = (\mathbf{W}^i)_{i\in\{Q,K,V,O\}}$ under randomness o. The provider publishes c_W once, before answering any query, and reuses it across all subsequent queries and across all circuit configurations that share these weights.
- $(\mathbf{Y}', \pi) \leftarrow \mathsf{zkDIET.Prove}(\mathsf{pk}, \mathsf{ck}, \mathsf{ck}_{\mathrm{glob}}, \mathbf{X}; \mathbf{W}, o)$: evaluate the pipeline on $\mathbf{X}$ outside the circuit to obtain the output $\mathbf{Y}'$, the non-linear operands lk and the auxiliary intermediates *aux*; invoke the commit-carrying backend, which returns a proof π_{cc} together with the proof-dependent commitments $\widetilde{\mathsf{cm}}_W$, $\widetilde{\mathsf{cm}}_{\mathsf{lk}}$, and $\widetilde{\mathsf{cm}}_{\mathsf{aux}}$ generated under ck; and produce a link proof π_{link} asserting that $\widetilde{\mathsf{cm}}_W$ and c_W open to the same weights, and that $\widetilde{\mathsf{cm}}_{\mathsf{lk}}$ and the operand commitment of the lookup argument open to the same lk. The output proof is $\pi := (\pi_{cc}, \widetilde{\mathsf{cm}}_W, \widetilde{\mathsf{cm}}_{\mathsf{lk}}, \widetilde{\mathsf{cm}}_{\mathsf{aux}}, \pi_{\mathrm{link}})$.
- $\{0,1\} \leftarrow \mathsf{zkDIET.Verify}(\mathsf{vk}, \mathbf{X}, \mathbf{Y}', c_W, \pi)$: accept only if π_{cc} verifies against the public instance, the link proof holds, and the lookup argument accepts.

The verifier's trust anchor is the circuit-independent c_W, not the per-proof commitments; the link proof π_{link} bridges the two, allowing responses across different circuit configurations to be traced to the same model.

Threat Model. The two parties mutually distrust each other. The provider is *malicious*: it may answer with the output of a cheaper or stale model, silently swap weights between queries or between circuit configurations, or fabricate intermediate values such as the pseudoinverse witness in order to make an incorrect output verify. The user is *honest but curious*: it follows the protocol but tries to learn the weights from everything it observes, namely the published commitment, the per-proof commitments, the challenge vectors, and the proof itself. Under this model we target two properties.

- *Inference integrity.* An accepted $\mathbf{Y}'$ is the result of evaluating, under the declared architecture, the very weights fixed by the published commitment c_W, and not an arbitrary value. Since c_W is issued once and is independent of the circuit, this binds every response the provider ever gives to a single model.
- *Model privacy in the proof.* The transcript $\left(c_W, \widetilde{\mathsf{cm}}_W, \widetilde{\mathsf{cm}}_{\mathsf{lk}}, \widetilde{\mathsf{cm}}_{\mathsf{aux}}, \pi_{cc}, \pi_{\mathrm{link}}\right)$ leaks nothing about $\mathbf{W}$ beyond what $\mathbf{Y}'$ itself reveals.

6.2 Security Definition

We keep the notation of Section 6.1 throughout.

Definition 1 (Zero-knowledge verifiable low-rank attention inference). zkDIET *is secure for* $\mathcal{F}^{\varepsilon}_{\mathsf{zkdiet}}$ *if it satisfies the following three properties.* Completeness*: an honest prover always convinces the verifier.* Soundness*: no PPT adversary can make the verifier accept an output* $\mathbf{Y}^* \notin \mathcal{F}^{\varepsilon}_{\mathsf{zkdiet}}(\mathbf{W}^*, \mathbf{X})$ *for the weights* $\mathbf{W}^*$ *bound by its own commitment* c^*_W*, except with negligible probability.* Zero-knowledge*: the proof transcript reveals nothing about the weights* $\mathbf{W}$ *beyond what the output* $\mathbf{Y}'$ *itself reveals; formally, there exists a PPT simulator that, given only* $(\mathbf{X}, \mathbf{Y}')$*, produces a transcript computationally indistinguishable from the real one. The formal probability statements and the Real/Ideal experiment are given in Appendix C.*

6.3 Security Analysis

Theorem 1 (informal). *Assume (i) the knowledge soundness and zero-knowledge of the CC-SNARK backend and of the CP-link proofs, (ii) the soundness and zero-knowledge of the lookup argument, (iii) the binding and hiding properties of the commitment scheme, and (iv) the random-oracle model for the Fiat–Shamir hash. Then* zkDIET *satisfies completeness, soundness, and zero-knowledge for* $\mathcal{F}^{\varepsilon}_{\mathsf{zkdiet}}$ *in the sense of Definition 1. Formal probability statements appear in Appendix C; we sketch the argument below.*

Soundness. Given an accepting transcript, knowledge soundness of the CC-SNARK yields a witness satisfying the relation of Figure 4; the binding of the commitments together with the link proof π_{link} ensures that the weights in this witness coincide with those bound by the published c_W, and the lookup argument guarantees that the non-linear operands $(\tilde{F}, \tilde{A}, \tilde{B})$ are correct evaluations of the quantized softmax. It remains to bound the probability that a witness passes the randomized checks while violating one of the underlying matrix identities. Each equality in lines 6–8, 13–15, and 23–26 of Figure 4 is a rank-one projection of a matrix identity; by Freivalds' bound, a violated identity passes its projected check with probability at most $1/|\mathbb{F}|$. Every matrix appearing in a projected identity falls into one of three categories: (i) public values ($\mathbf{X}$, $\mathbf{Y}'$); (ii) values committed before challenge derivation—the weights via $\widetilde{\mathsf{cm}}_W$, the intermediates $\mathbf{Q}, \mathbf{K}, \mathbf{V}, \tilde{A}^+, \tilde{F}\tilde{A}^+, \tilde{B}\mathbf{V}, \mathbf{Y}$ via $\widetilde{\mathsf{cm}}_{\mathsf{aux}}$, and the lookup indices and remainder limbs via the lookup operand commitment (see footnote 4); or (iii) values uniquely determined from committed data by exact in-circuit constraints—$\widetilde{\mathbf{Q}}$ and $\widetilde{\mathbf{K}}$ (segment averages of the committed $\mathbf{Q}, \mathbf{K}$, whose division remainders are range-checked and covered by the lookup operand commitment; see footnote 4) and the three pre-softmax products (each entry recovered from the committed lookup index and remainder-limb decomposition). Freivalds' bound therefore applies to each projected identity against adversarially chosen witnesses. Any residual discrepancy introduced by clamping or rounding during lookup-index

derivation is a quantization artifact absorbed by the ε-tolerant functionality $\mathcal{F}^{\varepsilon}_{\mathsf{zkdiet}}$. A union bound over the ten projected identities gives a total soundness error of $10/|\mathbb{F}| + \mathsf{negl}(\lambda)$.

Pseudoinverse tolerance. Lines 19–22 enforce the four Moore–Penrose conditions as element-wise range checks ($\|\cdot\|_{\max} \le \varepsilon$). Because these are exact in-circuit constraints, they contribute no additional soundness error; ε defines the acceptance set of $\mathcal{F}^{\varepsilon}_{\mathsf{zkdiet}}$. For $\varepsilon > 0$, perturbation bounds [23] give $\|Z^* - \tilde{A}^+\| \le O(\varepsilon\,\|\tilde{A}^+\|)$ provided $\sigma_{\min}(\tilde{A}) \ge \sigma_0 > 0$ (well-conditionedness assumption, consistent with our measurements on row-normalized softmax matrices). The output deviation is therefore at most $O(\varepsilon\,\|\tilde{F}\|\,\|\tilde{A}^+\|\,\|\tilde{B}V\|)$—precisely the slack $\mathcal{F}^{\varepsilon}_{\mathsf{zkdiet}}$ admits. Empirically, residuals remain below 4% of the quantization range (Section 7.1).

Zero-knowledge. The simulator $\mathcal{S}_1$ generates the CRS with its trapdoor and publishes commitments to zero, which are indistinguishable from the real ones by the hiding property. Given $(\mathbf{X}, \mathbf{Y}')$, $\mathcal{S}_2$ invokes the zero-knowledge simulators of the CC-SNARK, the link proof, and the lookup argument, and programs the random oracle so that the simulated challenges are consistent with the simulated commitments. Since every remaining transcript element is either public or simulated, the transcript reveals nothing about $\mathbf{W}$ beyond $\mathbf{Y}'$.

Out-of-Scope Attacks. While Definition 1 guarantees inference integrity and weight privacy within the proof system, several orthogonal threats fall outside the scope of this work. First, a user issuing many queries may attempt to reconstruct the model weights $\mathbf{W}$ from the returned outputs $\mathbf{Y}'$ alone, entirely without relying on the proof. Defenses against such black-box model extraction (e.g., rate limiting, output truncation, or watermarking) are studied independently and can be layered on top of our protocol.

Second, a commitment binds the provider to a single model, but does not guarantee model quality; an adversary could commit to a backdoored or low-accuracy model and continuously produce valid proofs for it. Attesting to the quality or fairness of the committed model requires auxiliary mechanisms (such as proving accuracy on a public labeled dataset), which we leave for future investigation.

7 Experiments

In this section, we evaluate the concrete proof generation costs of **zkDietFormer** and compare them against a standard self-attention baseline. Because our contribution targets the architectural cost of proving attention—i.e., the circuit structure rather than the learned weights—the evaluation focuses on the cost of the circuit itself. We use modified HuggingFace BERT and Nyströmformer configurations (with adjusted hidden dimensions and layer counts) to enable proving across multiple model sizes.

7.1 Implementation

We implement **zkDietFormer** in two parts: a Python-based witness generation pipeline and a Rust-based zero-knowledge proving circuit. Witness data (model weights and intermediate activations) are generated with Python 3.11.14, using PyTorch 2.9.1 and HuggingFace Transformers 4.57.3; our pipeline instruments the standard `BertForMaskedLM` and `NystromformerForMaskedLM` classes with forward hooks to capture per-layer activations. The Rust circuit is built on the arkworks library over the BLS12-381 elliptic curve and uses SHA3-256 (Keccak) for the Fiat–Shamir hash H of Section 5 and Poseidon for in-circuit hashing.

We use LegoGroth16 [4] as our commit-carrying SNARK (CC-SNARK) instantiation. Non-arithmetic operations—softmax exponentials, SigmoidGELU activation, and LayerNorm inverse square root—are evaluated using Lasso [22] lookup arguments. Theorem 1 requires the lookup argument to be zero-knowledge; Lasso admits standard zero-knowledge instantiations by blinding the committed operands, and our security analysis assumes such an instantiation. Our benchmarks use the non-hiding variant, whose costs are identical up to the blinding overhead.

Pseudoinverse Computation. As described in Section 5.3, the circuit verifies the prover-supplied $\tilde{A}^+$ against the Moore–Penrose conditions up to a bounded residual enforced via element-wise range checks. We empirically evaluated the maximum element-wise residual under a tiny-spec configuration (m=32 landmarks and 12-bit quantization). The observed maximum residuals for the verified conditions range from 1 to 161 relative to the full quantization range $[0, 4095]$. To accommodate these residuals, the circuit enforces bounds of $\varepsilon \in \{4, 64, 512\}$, providing a $3\times$ to $5\times$ safety margin above the empirical maximum errors, which strictly remain below 4% of the full range; the analysis of Section 6.3 conservatively takes ε to be the largest of these per-condition bounds.

Quantization. As noted in Section 4, we quantize all model computations following the R1CS-friendly quantization scheme of ZEN [10]. All weights and activations are converted to 12-bit asymmetric fixed-point representations, and the circuit precision parameter is set to $k_{\text{bits}} = 26$, providing sufficient headroom for accumulated multiplication errors across a full layer. Different operators use different fractional precisions to maintain numerical fidelity: LayerNorm uses 8-bit fractional precision (`FRAC_BITS` = 8); Softmax uses 16-bit fractional precision (`FRAC_BITS` = 16), reflecting the higher dynamic range required by the exponential function; and SigmoidGELU is evaluated via lookup tables of size $2^{16} = 65{,}536$ entries. For circuit efficiency we replace GELU with the lookup-friendly SigmoidGELU variant in all layers of both models; the definition and rationale are given in Appendix B.

7.2 Experimental Configuration

Hardware. All experiments run on a Mac Studio with an Apple M1 Ultra (20 cores) and 128 GB unified memory (CPU only; no GPU). Timing is CPU wall-clock via Rayon's thread pool; peak memory is the maximum RSS reported by the system `time` utility.

Model	L	d	h
Tiny	2	128	2
Mini	4	256	4
Small	4	512	8
Medium	8	512	8
Base	12	768	12

Table 1: Model configurations across the standard BERT family (Tiny to Base)

Baselines and Configurations. We compare two HuggingFace [29] architectures—`BertForMaskedLM` [9] ($\mathcal{O}(n^2 d)$ attention) and `NystromformerForMaskedLM` [30] ($\mathcal{O}(n)$ attention)—across five BERT-family configurations (Table 1). Both share identical (L, d, h) per scale, differing only in the attention kernel.

7.3 Results

All reported metrics are *per single layer*: each table entry is the average of the per-layer measurements across all L layers of the model (e.g., L=2 for Tiny). Timing values (setup, commit, prove, verify) and proof size are averaged over the L layers. Memory denotes the peak resident set size (RSS) observed during a single process that proves all L layers sequentially; because it is a process-wide maximum, it is *not* averaged but reported as a single value. We measure six sequence lengths $n \in \{128, 256, 512, 1024, 2048, 4096\}$ with a fixed landmark count m=32 for Nyström attention. Table 2 reports the full-pipeline sweep, which exhausts memory beyond n=1024; the attention-only sweep of Figure 5 extends to n=4096. This range is chosen to expose the asymptotic advantage of Nyström attention over standard self-attention: since the landmark count m is held fixed while n varies, the Nyström approximation scales as $\mathcal{O}(n)$ in both time and memory [30], whereas standard self-attention scales as $\mathcal{O}(n^2)$. For each circuit instance, we report constraint count, setup time, prove time, verify time, peak memory, and two commitment costs: the Groth16 committing time (G16) for committing to the committed witness slice—the weights and the intermediate matrices ($\mathbf{Q}$, $\mathbf{K}$, $\mathbf{V}$, $\tilde{A}^+$, $\tilde{F}\tilde{A}^+$, $\tilde{B}\mathbf{V}$, $\mathbf{Y}$), and the Lasso committing time for committing to the lookup operands. The global commitment to the model weights is a one-time cost independent of the number of layers and is therefore excluded from the per-layer metrics; in our measurements it takes approximately 965 ms. Runs that exceed available memory are reported as out-of-memory (OOM); these patterns are themselves informative results, as they indicate the practical scalability boundary of each circuit variant. We evaluate **zkDietFormer** by applying its two constituent techniques—low-rank (Nyström) attention and Freivalds-based matrix-multiplication verification—one at a time, first isolating each technique's individual contribution and then combining both to measure the advantage of the full scheme. In our benchmarks, we pass only cryptographic hashes of the input sequence $\mathbf{X}$ and the attention output $\mathbf{Y}'$ as the public instance, while supplying $\mathbf{X}$ and $\mathbf{Y}'$ themselves as witnesses. This approach avoids the prohibitive

setup cost incurred by large public-input vectors. Note that this is a deployment-time configuration; the protocol definition in Figure 3 presents $\mathbf{X}$ and $\mathbf{Y}'$ directly as public instances for clarity. Soundness is preserved under the collision resistance of the hash: the Fiat–Shamir transcript binds the hashed instance, so the fixed-before-challenge argument of Section 6.3 carries over.

Table 2: Full 2×2 factorial design—attention kernel (BERT / Nyström) × matrix-multiplication verification (Naive / Freivalds)—on the Tiny configuration (L=2, d=128, h=2). All metrics are per single layer (averaged over L layers). OOM entries (—) indicate that setup exhausted 128 GB; for these runs the Peak RAM column reports the observed process-wide maximum before termination. The G16 column reflects the extended committed slice (weights + $\mathbf{Q}, \mathbf{K}, \mathbf{V}, \tilde{A}^+, \tilde{F}\tilde{A}^+, \tilde{B}\mathbf{V}, \mathbf{Y}$).

				Commit (ms)					
n	Configuration	Constr. (M)	Setup (s)	G16	Lasso	Prove (s)	Verify (ms)	Proof (KB)	RAM (GB)
128	BERT-Naive	15.06	361.85	162.0	159.0	81.86	322.5	603.8	73.04
	BERT-Freivalds	4.97	164.49	190.0	164.5	33.41	319.0	603.8	29.75
	Nys-Naive	13.47	344.38	150.0	268.5	81.41	570.0	1,259.9	68.16
	Nys-Freivalds	4.86	160.46	185.0	258.5	35.94	604.0	1,259.9	29.53
256	BERT-Naive	39.22	912.39	624.5	266.0	345.44	418.5	726.8	112.13
	BERT-Freivalds	10.72	405.87	487.5	262.5	85.93	396.5	726.8	66.97
	Nys-Naive	26.49	697.46	275.5	312.0	187.33	664.5	1,346.7	112.18
	Nys-Freivalds	9.55	318.19	307.5	311.5	72.02	656.5	1,346.7	56.25
512	BERT-Naive	—	—	—	—	—	—	—	>113 (OOM)
	BERT-Freivalds	24.79	920.58	1,671.5	622.0	220.52	624.5	948.3	112.31
	Nys-Naive	—	—	—	—	—	—	—	>113 (OOM)
	Nys-Freivalds	18.93	640.69	557.5	466.0	152.70	789.0	1,475.2	109.10
1024	BERT-Naive	—	—	—	—	—	—	—	>113 (OOM)
	BERT-Freivalds	—	—	—	—	—	—	—	>113 (OOM)
	Nys-Naive	—	—	—	—	—	—	—	>113 (OOM)
	Nys-Freivalds	37.69	1,410.18	1,483.5	607.5	642.64	886.0	1,625.0	112.51

Low-Rank Attention is Essential for Asymptotic Scaling Fixing the verification method at naive and comparing the BERT-Naive vs. Nys-Naive rows of Table 2 (Tiny, L=2, d=128) isolates the attention kernel's effect. At n=128 the constraint gap is modest (15.06M vs. 13.47M, 1.1×), but by n=256 it widens to 39.22M vs. 26.49M (1.5×) with prove time diverging from 345.44 s to 187.33 s (1.8×). Both exhaust 128 GB at $n \geq 512$. The metrics on which Nys-Naive is worse than BERT-Naive—verify time, Lasso commit time, and proof size—reflect the overhead of the four Moore–Penrose checks and the three separate softmax factors; the additional proof elements, in particular, are independent of n.

Freivalds Removes the Constant Factor Holding the kernel at Nyström and comparing Nys-Naive vs. Nys-Freivalds isolates Freivalds' contribution. Across all sequence lengths, Freivalds reduces per-layer constraints by ∼2.8× (e.g.,

26.49M→9.55M at n=256), setup time by 2.2×, and prove time by 2.6×. This ratio is stable, confirming a uniform constant-factor improvement. Freivalds extends the feasibility boundary from n=256 to n=1024 (Table 2).

Freivalds Alone Cannot Substitute for Low-Rank Attention Freivalds yields comparable 3–3.7× constraint reductions on BERT (Table 2), confirming it is a generic optimization. Yet BERT-Freivalds still exhausts 128 GB at $n \geq 1024$ because $\mathcal{O}(n^2)$ complexity remains intact, whereas Nys-Freivalds reaches n=1024. The two optimizations are therefore complementary.

Combined Effect Comparing BERT-Naive with Nys-Freivalds in Table 2 quantifies the joint benefit. At n=256 the combined reduction reaches 4.1× in constraints, 2.9× in setup, and 4.8× in prove time, with peak memory dropping from 112.13 GB to 56.25 GB. BERT-Naive is infeasible at $n \geq 512$, while Nys-Freivalds extends to n=1024. Full MLP + LayerNorm cost breakdowns are deferred to Appendix B.

Multi-Spec Scaling Figure 5 isolates the cost of proving the attention equation itself—excluding global commitment generation, weight commitment via CC-SNARK, the CP-link proof between the global and weight commitments, and Freivalds-verified $\mathbf{Q}/\mathbf{K}/\mathbf{V}$ projection products—and compares BERT standard attention against Nyström attention across the full BERT model family.

The plot reports per-layer prove time (seconds) against sequence length n (linear axis) with prove time on a logarithmic axis. Each line terminates at the largest n feasible within our 128 GB memory budget. Red lines correspond to BERT and blue lines to Nyström. Within each color family, lighter shades represent smaller configurations and darker shades represent larger ones. Legend abbreviations: Bt/Ny = BERT/Nyström.

BERT's curves (red) rise steeply ($\mathcal{O}(n^2)$), while Nyström (blue) grows near-linearly ($\mathcal{O}(n)$). This gap translates directly into memory exhaustion: at Base (d=768), BERT caps at n=512 while Nyström reaches n=2048, a 4× extension. Medium is omitted as its per-layer cost matches Small (167.03 s vs. 167.48 s at n=512), confirming depth-invariance: per-layer cost depends on d and n, not L.

Quantization Fidelity. Table 3 evaluates 26-bit fixed-point quantization (k_{bits}=26) on pretrained uw-madison/nystromformer-512 at n=128 over 10 batches under full cascade evaluation (errors accumulated through all 12 layers; m=64 landmarks, the model default[5]). Top-1 agreement is 98.8% (90% at the `[MASK]` position), top-5 is 100%, cosine similarity 0.9975, confirming that 26-bit precision suffices for faithful ZK-SNARK proving.

[5] The circuit benchmarks in Table 2 use m=32.

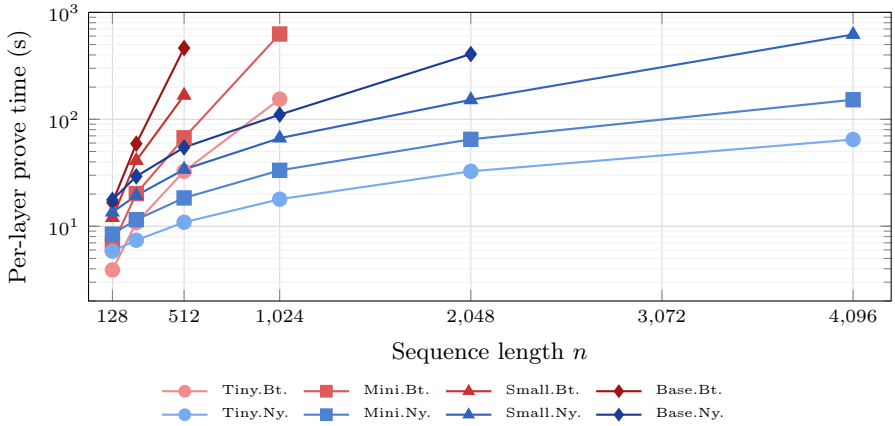


Fig. 5: Per-layer prove time for the attention equation only: BERT (red) vs. Nyström (blue), semi-log scale (linear n, logarithmic time).

Table 3: Quantization fidelity of 26-bit fixed-point arithmetic on a pretrained Nyströmformer (d=768, 12 layers, 12 heads, m=64 landmarks) at n=128.

Top-1 Agree. (%)		Top-5	Cosine	RMSE
All pos.	Mask pos.	All (%)	Sim.	(logit)
98.8	90.0	100.0	0.9975	0.378

7.4 Conclusion and Future Work

Combining Nyström low-rank attention with Freivalds verification yields multiplicative constraint savings and longer provable sequences; setup memory is the dominant bottleneck (cf. the OOM boundaries of Table 2). Once attention is optimized, MLP becomes the next memory frontier ($d \geq 512$ exceeds 128 GB; Appendix B).

Limitations. *(i)* CPU-only (Apple M1 Ultra); GPU proving would shift costs. *(ii)* Groth16 trusted setup is memory-bound; transparent SNARKs trade this for larger proofs. *(iii)* MLP sub-circuit remains memory-bound from $d \geq 256$ at longer sequences.

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

References

1. Beltagy, I., Peters, M.E., Cohan, A.: Longformer: The long-document transformer (2020), `https://arxiv.org/abs/2004.05150`

2. Brown, T.B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., Agarwal, S., Herbert-Voss, A., Krueger, G., Henighan, T., Child, R., Ramesh, A., Ziegler, D.M., Wu, J., Winter, C., Hesse, C., Chen, M., Sigler, E., Litwin, M., Gray, S., Chess, B., Clark, J., Berner, C., McCandlish, S., Radford, A., Sutskever, I., Amodei, D.: Language models are few-shot learners (2020), https://arxiv.org/abs/2005.14165
3. Campanelli, M., Faonio, A., Fiore, D., Querol, A., Rodríguez, H.: Lunar: a toolbox for more efficient universal and updatable zkSNARKs and commit-and-prove extensions. Cryptology ePrint Archive, Paper 2020/1069 (2020), https://eprint.iacr.org/2020/1069
4. Campanelli, M., Fiore, D., Querol, A.: LegoSNARK: Modular design and composition of succinct zero-knowledge proofs. Cryptology ePrint Archive, Paper 2019/142 (2019). https://doi.org/10.1145/3319535.3339820, https://eprint.iacr.org/2019/142
5. Chen, B.J., Waiwitlikhit, S., Stoica, I., Kang, D.: Zkml: An optimizing system for ml inference in zero-knowledge proofs. In: Proceedings of the Nineteenth European Conference on Computer Systems. p. 560–574. EuroSys '24, Association for Computing Machinery, New York, NY, USA (2024). https://doi.org/10.1145/3627703.3650088, https://doi.org/10.1145/3627703.3650088
6. Child, R., Gray, S., Radford, A., Sutskever, I.: Generating long sequences with sparse transformers (2019), https://arxiv.org/abs/1904.10509
7. Choromanski, K., Likhosherstov, V., Dohan, D., Song, X., Gane, A., Sarlos, T., Hawkins, P., Davis, J., Mohiuddin, A., Kaiser, L., Belanger, D., Colwell, L., Weller, A.: Rethinking attention with performers (2022), https://arxiv.org/abs/2009.14794
8. Cong, M., Yuen, T.H., Yiu, S.M.: zkmatrix: Batched short proof for committed matrix multiplication. In: Proceedings of the 19th ACM Asia Conference on Computer and Communications Security. p. 289–305. ASIA CCS '24, Association for Computing Machinery, New York, NY, USA (2024). https://doi.org/10.1145/3634737.3645003, https://doi.org/10.1145/3634737.3645003
9. Devlin, J., Chang, M.W., Lee, K., Toutanova, K.: Bert: Pre-training of deep bidirectional transformers for language understanding (2019), https://arxiv.org/abs/1810.04805
10. Feng, B., Qin, L., Zhang, Z., Ding, Y., Chu, S.: ZEN: An optimizing compiler for verifiable, zero-knowledge neural network inferences. Cryptology ePrint Archive, Paper 2021/087 (2021), https://eprint.iacr.org/2021/087
11. Freivalds, R.: Probabilistic machines can use less running time. In: IFIP Congress (1977), https://api.semanticscholar.org/CorpusID:45405368
12. Ghodsi, Z., Gu, T., Garg, S.: Safetynets: Verifiable execution of deep neural networks on an untrusted cloud. In: Guyon, I., Luxburg, U.V., Bengio, S., Wallach, H., Fergus, R., Vishwanathan, S., Garnett, R. (eds.) Advances in Neural Information Processing Systems. vol. 30. Curran Associates, Inc. (2017), https://proceedings.neurips.cc/paper_files/paper/2017/file/6048ff4e8cb07aa60b6777b6f7384d52-Paper.pdf
13. Groth, J.: On the size of pairing-based non-interactive arguments. In: Fischlin, M., Coron, J.S. (eds.) Advances in Cryptology – EUROCRYPT 2016. pp. 305–326. Springer Berlin Heidelberg, Berlin, Heidelberg (2016)
14. Hendrycks, D., Gimpel, K.: Gaussian error linear units (GELUs) (2016), https://arxiv.org/abs/1606.08415

15. Jang, B., Jeong, G., Kwon, H., Oh, H., Kim, J.: VECTIS: Efficient batching framework for group-based CP-SNARKs. Cryptology ePrint Archive, Paper 2024/1405 (2024), https://eprint.iacr.org/2024/1405
16. Korec, I., Wiedermann, J.: Deterministic verification of integer matrix multiplication in quadratic time. In: Geffert, V., Preneel, B., Rovan, B., Štuller, J., Tjoa, A.M. (eds.) SOFSEM 2014: Theory and Practice of Computer Science. pp. 375–382. Springer International Publishing, Cham (2014)
17. Lee, S., Ko, H., Kim, J., Oh, H.: vcnn: Verifiable convolutional neural network based on zk-snarks. IEEE Transactions on Dependable and Secure Computing **21**(4), 4254–4270 (2024). https://doi.org/10.1109/TDSC.2023.3348760
18. Liu, T., Xie, X., Zhang, Y.: zkcnn: Zero knowledge proofs for convolutional neural network predictions and accuracy. In: Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security. p. 2968–2985. CCS '21, Association for Computing Machinery, New York, NY, USA (2021). https://doi.org/10.1145/3460120.3485379, https://doi.org/10.1145/3460120.3485379
19. Meta: Llama 4 Model Card. GitHub repository documentation (2024), https://github.com/meta-llama/llama-models/blob/main/models/llama4/MODEL_CARD.md, accessed: 2025-12-31
20. OpenAI: GPT-5 Model. OpenAI API Documentation (2025), https://platform.openai.com/docs/models/gpt-5, accessed: 2025-12-31
21. Rae, J.W., Potapenko, A., Jayakumar, S.M., Lillicrap, T.P.: Compressive transformers for long-range sequence modelling (2019), https://arxiv.org/abs/1911.05507
22. Setty, S., Thaler, J., Wahby, R.: Unlocking the lookup singularity with Lasso. Cryptology ePrint Archive, Paper 2023/1216 (2023), https://eprint.iacr.org/2023/1216
23. Stewart, G.W., Sun, J.g.: Matrix Perturbation Theory. Academic Press (1990)
24. Sun, H., Li, J., Zhang, H.: zkllm: Zero knowledge proofs for large language models (2024), https://arxiv.org/abs/2404.16109
25. Thaler, J.: Time-optimal interactive proofs for circuit evaluation. In: Canetti, R., Garay, J.A. (eds.) Advances in Cryptology – CRYPTO 2013. pp. 71–89. Springer Berlin Heidelberg, Berlin, Heidelberg (2013)
26. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, L., Polosukhin, I.: Attention is all you need (2023), https://arxiv.org/abs/1706.03762
27. Wang, S., Li, B.Z., Khabsa, M., Fang, H., Ma, H.: Linformer: Self-attention with linear complexity (2020), https://arxiv.org/abs/2006.04768
28. Weng, J., Weng, J., Tang, G., Yang, A., Li, M., Liu, J.N.: pvcnn: Privacy-preserving and verifiable convolutional neural network testing (2023), https://arxiv.org/abs/2201.09186
29. Wolf, T., Debut, L., Sanh, V., Chaumond, J., Delangue, C., Moi, A., Cistac, P., Rault, T., Louf, R., Funtowicz, M., Davison, J., Shleifer, S., von Platen, P., Ma, C., Jernite, Y., Plu, J., Xu, C., Scao, T.L., Gugger, S., Drame, M., Lhoest, Q., Rush, A.: Transformers: State-of-the-art natural language processing. In: Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations. pp. 38–45. Association for Computational Linguistics (2020)
30. Xiong, Y., Zeng, Z., Chakraborty, R., Tan, M., Fung, G., Li, Y., Singh, V.: Nyströmformer: A nyström-based algorithm for approximating self-attention (2021), https://arxiv.org/abs/2102.03902

31. Zhang, Y., Zheng, M., Chen, X., Hu, J., Shi, W., Ju, L., Solihin, Y., Lou, Q.: zkvc: Fast zero-knowledge proof for private and verifiable computing (2025), https://arxiv.org/abs/2504.12217

A Newton–Schulz Iteration

The prover computes the iterative approximation Z^* of the pseudoinverse $\tilde{A}^+$ using the Newton–Schulz iteration detailed in Algorithm 2. The circuit does not execute this iteration; instead, it verifies the result via the four Moore–Penrose conditions (Section 5.3), making the iteration an off-circuit reference procedure.

Algorithm 2 Compute Moore–Penrose inverse

Input: selected landmark matrix $\tilde{A} \in \mathbb{F}^{m \times m}$
Output: approximate pseudoinverse $\tilde{A}^+ \in \mathbb{F}^{m \times m}$
ComputeMPInverse($\tilde{A}$):

$Z_0 \leftarrow \tilde{A}^\top / \left(\|\tilde{A}\|_1 \cdot \|\tilde{A}\|_\infty \right)$
Set T (e.g., $T = 6$ iterations)
for $t \in \{0, \ldots, T-1\}$ **do**
  $Z_{t+1} \leftarrow \frac{1}{4} Z_t \Big(13I - \tilde{A} Z_t \big(15I - \tilde{A} Z_t (7I - \tilde{A} Z_t) \big) \Big)$
endfor
$\tilde{A}^+ \leftarrow Z_T$
return $\tilde{A}^+$

B MLP + LayerNorm Circuit Costs

A complete transformer layer consists of two sub-circuits: attention and the MLP + LayerNorm block. While the main text focuses on the attention sub-circuit—the target of our **zkDietFormer** optimizations—we additionally measured the cost of proving the MLP + LayerNorm sub-circuit, whose constraint count is the same for BERT and Nyströmformer because the MLP and LayerNorm dimensions depend only on n and d, not on the attention mechanism.

Activation Function Replacement. One modification required by the end-to-end measurements reported in this appendix concerns the GELU activation used in the MLP block. Both BERT and Nyströmformer use GELU [14] as their default activation, typically implemented via the approximation $\mathrm{GELU}(x) \approx 0.5\,x\big(1 + \tanh\big[\sqrt{2/\pi}\,(x + 0.044715\,x^3)\big]\big)$. We replace GELU in both models with SigmoidGELU, defined as

$$\mathrm{SigmoidGELU}(x) = x \cdot \sigma(1.702\,x),$$

where $\sigma(\cdot)$ denotes the logistic sigmoid. This alternative, originally noted by Hendrycks and Gimpel [14] as a close approximation to GELU, is substantially more circuit-friendly because the sigmoid function can be evaluated via a single lookup table, whereas the tanh-based form requires a more involved computational procedure. The replacement is applied to all layers in both models.

Table 4 reports per-layer average MLP + LayerNorm costs for the Tiny configuration (L=2); all timing and memory figures are the mean of the BERT and Nyström runs. For larger hidden dimensions the MLP block exhausted the memory budget—at $d \geq 512$ setup ran out of memory at all measured sequence lengths, and at $d \geq 256$ at longer sequences—so we report the Tiny configuration only. These measurements show that the MLP + LayerNorm sub-circuit and the attention sub-circuit contribute comparable costs at n=1024: per-layer MLP + LayerNorm proving takes 517.21 s versus 642.64 s for Nys-Freivalds attention. The MLP + LayerNorm block consumes 112.70 GB peak RAM at n=1024, making it a co-bottleneck alongside attention for memory-constrained environments.

For reference, proving a complete Tiny transformer layer (attention + MLP + LayerNorm) at n=1024 requires approximately 2,290.28 s of per-layer setup and 1,159.85 s of per-layer proving (∼58 min total per layer). We report these measurements primarily as a baseline for future work on scalable ZK proving of the full transformer layer.

Table 4: **MLP + LayerNorm** costs for the Tiny configuration (L=2, d=128).

			Commit (ms)					
n	Constr. (M)	Setup (s)	G16	Lasso	Prove (s)	Verify (ms)	Proof (KB)	RAM (GB)
128	0.98	95.10	116.0	150.3	28.69	287.5	520.7	26.46
256	1.96	189.64	217.8	172.3	58.84	309.5	572.8	53.31
512	3.92	385.39	438.0	295.3	138.99	409.5	654.6	101.55
1024	7.84	880.10	1,128.3	426.5	517.21	477.3	746.1	112.70

C Formal Security Definitions

We state the formal probability conditions for the three security properties of Definition 1. Notation follows Section 6.1.

Completeness. For every $\mathbf{W} = (\mathbf{W}^i)_{i \in \{Q,K,V,O\}}$ with each $\mathbf{W}^i \in \mathbb{F}^{d \times d}$ and every $\mathbf{X} \in \mathbb{F}^{n \times d}$, for $\mathsf{ck}_{\mathrm{glob}} \leftarrow \mathsf{KeyGen}(1^\lambda, N)$, $(\mathsf{ck}, \mathsf{pk}, \mathsf{vk}) \leftarrow \mathsf{Setup}(1^\lambda, \mathcal{R}_{\mathsf{zkdiet}})$, $c_W \leftarrow \mathsf{Commit}(\mathsf{ck}_{\mathrm{glob}}, \mathbf{W}; o)$ and $(\mathbf{Y}', \pi) \leftarrow \mathsf{Prove}(\mathsf{pk}, \mathsf{ck}, \mathsf{ck}_{\mathrm{glob}}, \mathbf{X}; \mathbf{W}, o)$,

$$\Pr\big[\mathsf{Verify}(\mathsf{vk}, \mathbf{X}, \mathbf{Y}', c_W, \pi) = 1\big] = 1.$$

Soundness. For any PPT adversary $\mathcal{A}$, there exists a PPT extractor $\mathcal{E}$ such that

$$\Pr\left[\begin{array}{c} \mathsf{Verify}(\mathsf{vk}, \mathbf{X}, \mathbf{Y}^*, c_W^*, \pi^*) = 1 \\ \wedge \;\; c_W^* = \mathsf{Commit}(\mathsf{ck}_{\mathrm{glob}}, \mathbf{W}^*; o^*) \\ \wedge \;\; \mathbf{Y}^* \notin \mathcal{F}^{\varepsilon}_{\mathsf{zkdiet}}(\mathbf{W}^*, \mathbf{X}) \end{array} \middle| \begin{array}{l} \mathsf{ck}_{\mathrm{glob}} \leftarrow \mathsf{KeyGen}(1^\lambda, N) \\ (\mathsf{ck}, \mathsf{pk}, \mathsf{vk}) \leftarrow \mathsf{Setup}(1^\lambda, \mathcal{R}_{\mathsf{zkdiet}}) \\ (c_W^*, \mathbf{X}, \mathbf{Y}^*, \pi^*) \leftarrow \mathcal{A}(\mathsf{ck}_{\mathrm{glob}}, \mathsf{ck}, \mathsf{pk}, \mathsf{vk}) \\ (\mathbf{W}^*, o^*) \leftarrow \mathcal{E}^{\mathcal{A}}(\mathsf{trans}_{\mathcal{A}}) \end{array}\right] \leq \mathsf{negl}(\lambda).$$

Zero-knowledge. Write $\mathsf{Eval}(\mathbf{W}, \mathbf{X})$ for the deterministic off-circuit evaluation of Algorithm 1. There exists a PPT simulator $\mathcal{S} = (\mathcal{S}_1, \mathcal{S}_2)$ such that for every $\mathbf{W}$ and every PPT adversary $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2)$, the two experiments below are computationally indistinguishable (barred symbols denote simulator-produced values):

$\underline{\mathsf{Real}_{\mathcal{A},W}(\lambda)}$	$\underline{\mathsf{Ideal}_{\mathcal{A},\mathcal{S},W}(\lambda)}$
$\mathsf{ck}_{\mathrm{glob}} \leftarrow \mathsf{KeyGen}(1^\lambda, N)$	$(\overline{\mathsf{ck}}_{\mathrm{glob}}, \overline{\mathsf{ck}}, \overline{\mathsf{pk}}, \overline{\mathsf{vk}}, \bar{c}_W, \mathsf{td}) \leftarrow \mathcal{S}_1(1^\lambda, N, \mathcal{R}_{\mathsf{zkdiet}})$
$(\mathsf{ck}, \mathsf{pk}, \mathsf{vk}) \leftarrow \mathsf{Setup}(1^\lambda, \mathcal{R}_{\mathsf{zkdiet}})$	
$o \xleftarrow{\$} \mathcal{O}; \;\; c_W \leftarrow \mathsf{Commit}(\mathsf{ck}_{\mathrm{glob}}, \mathbf{W}; o)$	
$(\mathbf{X}, st) \leftarrow \mathcal{A}_1(\mathsf{ck}_{\mathrm{glob}}, \mathsf{ck}, \mathsf{pk}, \mathsf{vk}, c_W)$	$(\mathbf{X}, st) \leftarrow \mathcal{A}_1(\overline{\mathsf{ck}}_{\mathrm{glob}}, \overline{\mathsf{ck}}, \overline{\mathsf{pk}}, \overline{\mathsf{vk}}, \bar{c}_W)$
$\mathbf{Y}' := \mathsf{Eval}(\mathbf{W}, \mathbf{X})$	$\mathbf{Y}' := \mathsf{Eval}(\mathbf{W}, \mathbf{X})$
$\pi \leftarrow \mathsf{Prove}(\mathsf{pk}, \mathsf{ck}, \mathsf{ck}_{\mathrm{glob}}, \mathbf{X}; \mathbf{W}, o)$	$\overline{\pi} \leftarrow \mathcal{S}_2(\mathsf{td}, \mathbf{X}, \mathbf{Y}')$
return $\mathcal{A}_2(st, \mathbf{Y}', \pi)$	**return** $\mathcal{A}_2(st, \mathbf{Y}', \overline{\pi})$

$$\left|\Pr\big[\mathsf{Real}_{\mathcal{A},W}(\lambda) = 1\big] - \Pr\big[\mathsf{Ideal}_{\mathcal{A},\mathcal{S},W}(\lambda) = 1\big]\right| \leq \mathsf{negl}(\lambda).$$